

Smart Surveillance and Motion Analysis using Computer Vision

A modular command-line pipeline for object detection, persistent tracking, motion estimation, event logging and analytics on recorded video.

Student Name	Vedansh Taparia
Registration Number	24BAI10419
Course	Computer Vision
Institution	VIT Bhopal University
GitHub Repository	https://github.com/10Vedansh/smart-surveillance-cv
Submission Date	September 2026

Project scope: recorded-video analysis only. The system does not perform facial recognition, identity inference, suspiciousness classification, live CCTV deployment, or automatic security decisions.

1. Abstract

Manual inspection of surveillance footage can require repeated frame-by-frame review to locate moving objects and understand when events occurred. This project implements a modular Computer Vision pipeline that accepts a recorded video through the command line, preprocesses frames, detects and tracks objects using a pretrained YOLO model, estimates motion using frame differencing and Farneback dense optical flow, and records motion-associated events. The resulting data are exported as a CSV event log and a JSON run summary, while an annotated MP4 can be generated for visual review.

The design separates input handling, image preprocessing, detection and tracking, motion analysis, event logging, and analytics into independent modules. Runtime parameters such as confidence, motion thresholds, output width, cooldown and frame limits are configurable through the command line. Automated tests cover deterministic image-processing, motion, logging, analytics, video-reader and command-line behavior.

The project is intentionally scoped as an academic recorded-video analysis system. It does not attempt to identify individuals, infer intent, decide whether a person is suspicious, or provide real-time security decisions.

2. Introduction

Computer Vision combines image processing and visual inference techniques to extract useful information from images and video. Surveillance footage provides a practical setting in which multiple CV concepts can be integrated into one pipeline. This project demonstrates that integration through preprocessing, object detection, multi-object tracking, motion estimation, structured event logging and quantitative run-level analytics.

The implementation is terminal-based rather than GUI-driven. This makes the processing workflow reproducible and allows the same analysis to be repeated with different input videos and parameter settings.

3. Problem Statement

Manual review of surveillance footage is time-consuming because an operator must repeatedly inspect frames to locate moving objects and understand when events occurred. The proposed system addresses this by automatically processing stored video, detecting and tracking objects, estimating motion, and generating machine-readable event records.

4. Objectives

- Process a stored surveillance video frame by frame.
- Apply image preprocessing techniques before visual analysis.
- Detect and track objects using a pretrained YOLO model.
- Estimate motion using frame differencing and dense optical flow.
- Log motion-associated object observations with timestamps and confidence values.
- Produce summary analytics and an annotated output video without requiring a GUI.

5. Scope and Boundaries

In Scope	Out of Scope
Recorded MP4/compatible video input	Facial recognition or identity inference
Object detection and persistent tracking	Automatic judgment of whether a person is suspicious
Motion estimation and event logging	Live CCTV deployment

In Scope	Out of Scope
Command-line execution and structured outputs	Cloud deployment and notification services
Automated tests for deterministic modules	Training a new detection model

6. Functional Requirements

ID	Requirement
FR-1	Accept a compatible video path through the command line.
FR-2	Validate video availability and readable properties.
FR-3	Resize frames while preserving aspect ratio.
FR-4	Apply Gaussian denoising, CLAHE contrast enhancement and Canny edge extraction.
FR-5	Detect and track objects using a pretrained YOLO model.
FR-6	Estimate motion using frame differencing and dense optical flow.
FR-7	Log motion-associated tracked-object events with confidence and motion metrics.
FR-8	Generate an annotated output video unless disabled.
FR-9	Generate CSV and JSON analytics.
FR-10	Expose processing parameters through command-line options.

7. Non-Functional Requirements

Category	Implementation
Performance	Configurable processing width and optional frame limit.
Reliability	Clear exceptions for invalid input/output initialization and resource release.
Maintainability	Major responsibilities are isolated into separate modules.
Usability	Terminal execution with documented commands and --help support.
Resource efficiency	Frames are resized before expensive analysis.
Testability	Deterministic modules are covered by pytest.
Observability	Console summaries and machine-readable CSV/JSON reports.

8. Hardware and Software Requirements

Item	Requirement / Recommendation
Language	Python 3.11+
RAM	8 GB or more recommended
Compute	CPU supported; compatible GPU may accelerate YOLO inference
Libraries	OpenCV, NumPy, Ultralytics, pytest
Input	Recorded MP4/compatible video
Outputs	Annotated MP4, events.csv, summary.json

9. Methodology

The system follows a sequential modular pipeline. The main program orchestrates the modules while each module owns a focused responsibility. The pipeline begins with video validation and frame extraction, continues through preprocessing and visual inference, and ends with structured event records and run-level analytics.

9.1 Input and Preprocessing

- VideoReader validates the input path and exposes video properties such as width, height and frame rate.
- ImageProcessor resizes frames while preserving aspect ratio.
- Gaussian smoothing is used for denoising.
- CLAHE is used for local contrast enhancement.
- Canny edge extraction is available as a classical image-processing feature.

9.2 Detection and Tracking

A pretrained YOLO model is used to detect objects in the processed frames. Persistent tracking associates detections across frames using track IDs. The system records object class and confidence along with the track identifier.

9.3 Motion Analysis

- Frame differencing compares the current frame with the previous frame and produces a thresholded changed-pixel mask.
- Farneback dense optical flow estimates motion vectors between consecutive frames.
- The resulting motion measurements provide complementary information: changed-pixel count is simple and interpretable, while mean optical-flow magnitude provides a motion-strength measure.

9.4 Event Logging

When a tracked object is associated with detected motion, the event logger records the observation. A cooldown mechanism is used to reduce repeated records for the same track on consecutive frames while retaining events that occur later.

9.5 Analytics

After frame processing, the analytics module aggregates the event log and creates summary.json. The summary includes frames processed, elapsed time, event count, unique tracking IDs, class counts and average motion/confidence values.

10. System Architecture

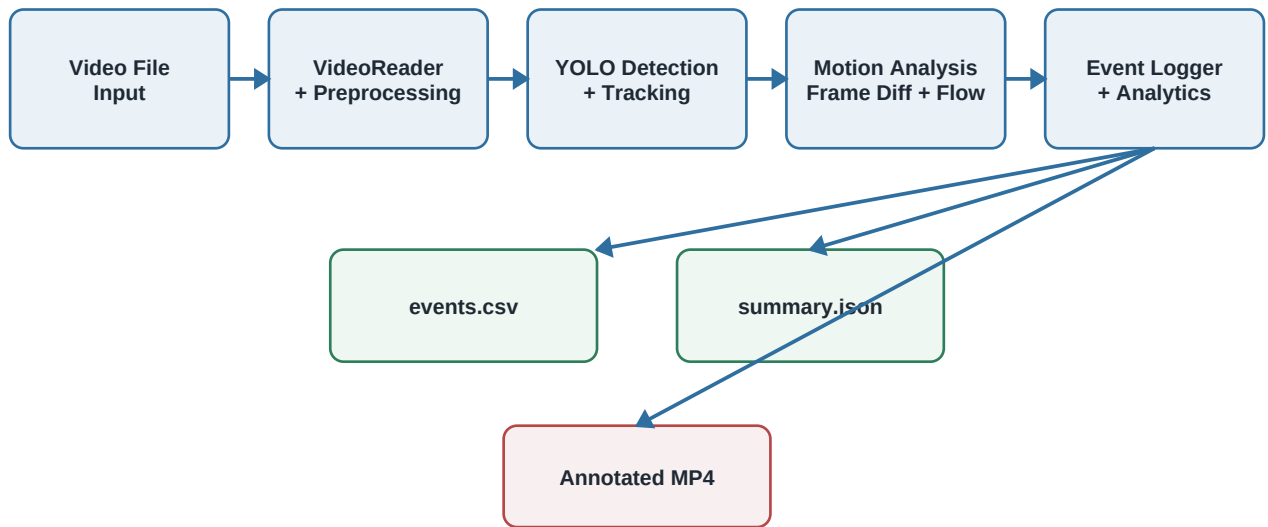


Figure 1. Modular architecture of the Smart Surveillance pipeline.

The architecture separates preprocessing, detection/tracking, motion analysis, event logging and analytics. The design documentation specifies that the pipeline produces an annotated MP4, events.csv and summary.json.

11. Workflow

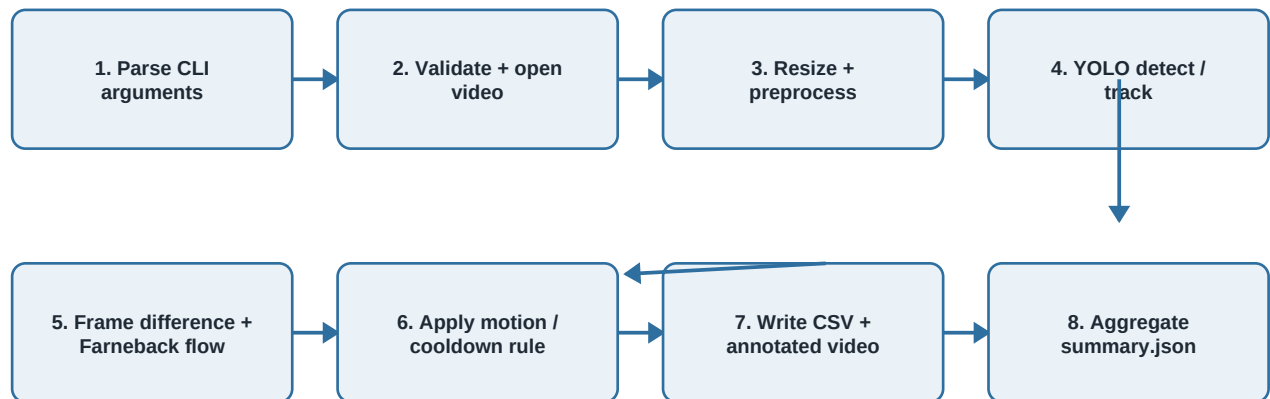


Figure 2. Processing workflow from command-line execution to structured outputs.

The workflow consists of command-line parsing, input validation, frame preprocessing, YOLO tracking, motion estimation, cooldown-based event logging, output generation and final analytics aggregation.

12. Use Case View

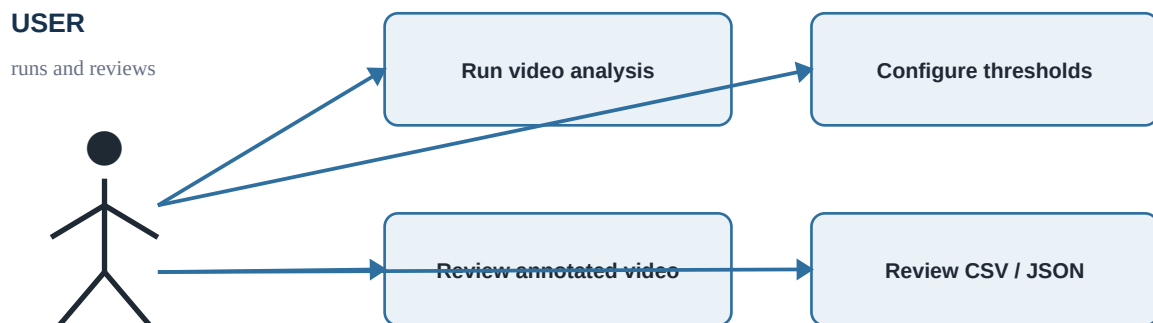


Figure 3. User-level use cases.

- Run video analysis from the terminal.
- Configure model and motion thresholds.
- Review the annotated video.

- Review the CSV event log and JSON summary.

13. Sequence of Operations

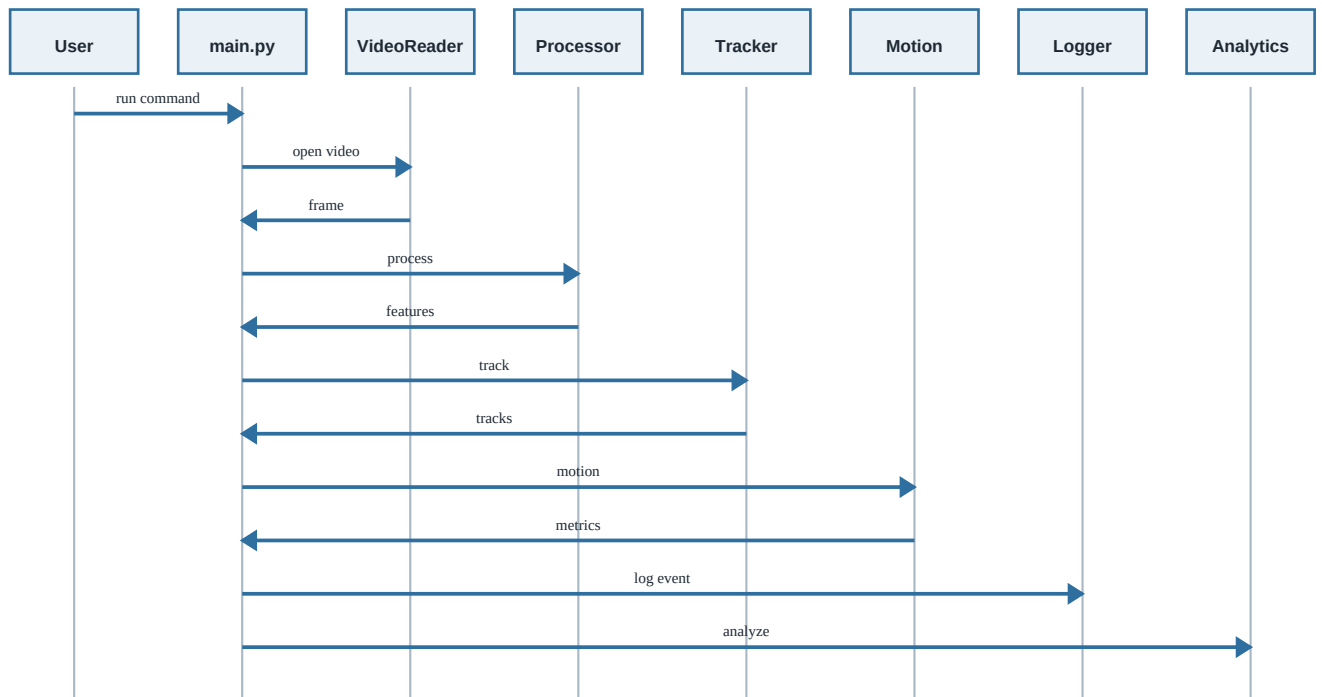


Figure 4. Simplified sequence diagram for one processing run.

14. Component and Class Design

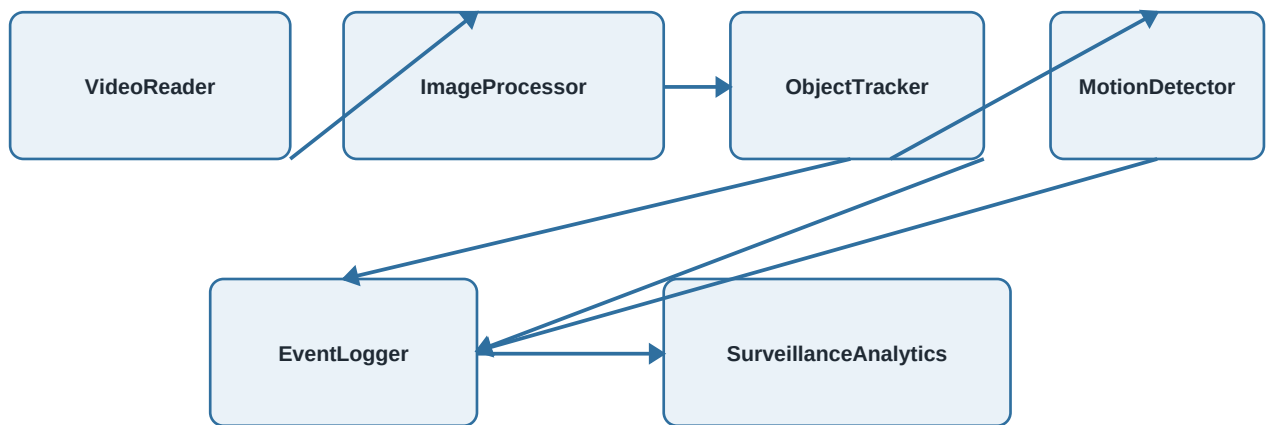


Figure 5. Main component/class relationships.

Component	Primary Responsibility
VideoReader	Open the input video, expose properties, read frames and release resources.
ImageProcessor	Resize, grayscale, denoise, enhance contrast, extract edges and return processed features.
ObjectTracker	Run YOLO tracking and expose persistent object tracks.
MotionDetector	Calculate frame-difference and optical-flow motion measurements.
EventLogger	Write motion-associated tracked-object observations to CSV.
SurveillanceAnalytics	Aggregate event data and export run-level JSON analytics.

15. Data and Storage Design

The current project does not require a relational database. A lightweight CSV file is used as the event store and a JSON file is used for run-level summary metrics.

Field	Meaning
timestamp	Event creation time
frame_number	Frame where the event was recorded
event_type	Currently recorded as Motion Detected
object_id	Persistent tracker ID
object_class	YOLO class name
confidence	Detection/tracking confidence
motion_pixels	Number of changed pixels in the motion mask

Field	Meaning
mean_flow	Mean optical-flow magnitude

The summary JSON stores run-level metrics such as frames processed, elapsed time, event count, unique IDs, class counts and average motion/confidence values.

16. Implementation Details

The project is organized as independent Python packages. The main entry point coordinates the processing loop, while configuration and domain responsibilities are kept separate. The README documents the repository structure, setup commands, execution modes and output files.

16.1 Technology Stack

Technology	Role in Project
Python	Application and module implementation
OpenCV	Video I/O, frame processing, drawing and optical-flow/image operations
NumPy	Numerical array operations for image and motion processing
Ultralytics YOLO	Pretrained object detection and persistent tracking
pytest	Automated tests for deterministic modules
Git / GitHub	Version control and public project repository

16.2 Runtime Configuration

The command-line workflow supports configuration of the input video, output directory, model, processing width, confidence threshold, motion thresholds, cooldown and optional maximum frame count. This allows the same implementation to be reused for validation and experimentation.

16.3 Example Commands

```
python main.py --help

python main.py --input input/test_video.mp4 \
  --output-dir output --max-frames 30

python main.py --input input/test_video.mp4 \
  --output-dir output

python main.py --input input/test_video.mp4 \
  --output-dir output --no-video
```

The repository includes yolo11n.pt, so the documented workflow does not require a separate model download for the included model file.

17. Design Decisions and Rationale

- Command line instead of GUI: supports terminal-based execution and reproducible batch processing.
- YOLO tracking: supplies object classes, confidence values and persistent IDs without implementing a tracker from scratch.
- Two motion measurements: combines an interpretable changed-pixel measure with optical-flow magnitude.
- Cooldown logging: reduces duplicate records while preserving later motion events.
- CSV + JSON: lightweight, portable formats that are sufficient for the current single-run project scope.
- Lazy model dependency handling in deterministic tests: keeps basic tests independent of heavy model inference.

18. Testing and Validation

The test strategy separates deterministic image-processing, motion, logging and analytics logic from YOLO inference. This permits automated tests to execute without requiring model inference. The documented suite checks frame resizing, grayscale conversion, CLAHE output, edge extraction, synthetic frame-difference/optical-flow behavior, event CSV creation, analytics aggregation, temporary-video validation and command-line parser behavior.

18.1 Test Execution

```
python -m pytest -q
```

The project also provides an end-to-end validation command using the included test video and model.

18.2 Observed Quick Validation Run

Metric	Observed value
Frames processed	30
Processing time	24.18 s
Total logged events	16
Unique tracked IDs	10
Average confidence	0.732
Average motion pixels	101146.25
Average optical-flow magnitude	2.653
Object classes	car (2), person (12), traffic light (2)

18.3 Observed Full Run

Metric	Observed value
Frames processed	186
Processing time	65.43 s
Total logged events	101
Unique tracked IDs	23
Average confidence	0.675
Average motion pixels	150243.95
Average optical-flow magnitude	4.301
Object classes	person (72), traffic light (13), car (13), bicycle (3)

All three expected end-to-end outputs were reported as created successfully: output/events.csv, output/summary.json and output/surveillance_output.mp4.

The observed metrics are environment- and input-dependent. Detection/tracking results can change with the pretrained model, video, confidence threshold and hardware; therefore these values are reported as validation observations rather than universal benchmarks.

19. Output Analysis

The project produces three primary artifacts from a successful run.

Output	Purpose
surveillance_output.mp4	Annotated video containing tracked bounding boxes, IDs and motion status/metrics.
events.csv	Structured motion-associated tracked-object event records.
summary.json	Run-level analytics including frame count, runtime, event count, IDs, class counts and averages.

The output design supports both human review and machine-readable analysis. The annotated MP4 provides visual evidence of detections and motion, while CSV/JSON allow downstream inspection without parsing video frames.

20. Evaluation Summary

The implementation demonstrates the integration of multiple Computer Vision concepts rather than relying on a single inference call. The project includes classical image preprocessing, temporal motion estimation, dense optical flow, pretrained object detection, persistent tracking, structured event logging, analytics and automated testing.

Area	Evidence in Project
CV relevance	Preprocessing, frame differencing, optical flow, object detection and tracking.
Modularity	Separate input, preprocessing, detection, tracking, motion, logging and analytics modules.
Configurability	Command-line options for input/output and analysis thresholds.
Testing	pytest coverage of deterministic modules plus end-to-end validation procedure.
Documentation	README, statement, requirements, design and testing documentation.
Reproducibility	Documented setup and run commands; included model file and test video.

21. Limitations

- The system processes recorded video rather than live CCTV streams.
- Detection and tracking quality depends on the pretrained YOLO model, input video, confidence threshold and available hardware.
- Motion estimation can respond to scene changes and camera/background movement; the project does not claim semantic understanding of intent.
- The system does not identify individuals or infer identity.
- The project does not train a custom detection model.
- A CSV/JSON file is sufficient for the current academic scope but is not a replacement for a production surveillance data platform.

22. Future Scope

- Support live camera streams while preserving the same modular architecture.
- Add configurable region-of-interest rules for scene-specific analysis.
- Introduce richer event categories based on motion trajectories and spatial rules.
- Add a database or dashboard for multi-run analytics when the project scope expands.
- Evaluate detection/tracking performance on a labeled benchmark dataset rather than relying only on run-level observations.
- Add controlled notification mechanisms only as a separate, explicitly configured system component.

23. Conclusion

Smart Surveillance and Motion Analysis using Computer Vision provides a complete academic Computer Vision pipeline for recorded-video analysis. It combines preprocessing, object detection, persistent tracking, frame differencing, Farneback optical flow, event logging and analytics in a modular design. The use of structured outputs and automated tests makes the implementation suitable for demonstration, experimentation and further extension.

The project remains intentionally bounded: it analyzes supplied video and reports measurable visual events; it does not perform facial recognition, identity inference, suspiciousness classification or autonomous security decisions.

24. References and Project Resources

- OpenCV — computer vision and video-processing library used by the implementation.
- Ultralytics YOLO — pretrained detection/tracking framework used by the project.
- Python — implementation language.
- pytest — automated testing framework.
- Project repository: <https://github.com/10Vedansh/smart-surveillance-cv>

End of Report